PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING

10 Different Beginner-Level Case Studies with Solutions

Name: **BALIBOYINA NIHITHA**

Roll Number: 26000858

Branch / Section: CSM / 1

Course: Python Programming

Institution: Dr. RVR & NRI Institute of Technology

Prepared as an original beginner-level assignment using basic Python concepts suitable for first-year B.Tech students.

Note: Rates, fees, grades, and other rules are educational examples for this assignment, not official institutional or commercial policies.

Contents

1. Simple Interest Calculator

2. Temperature Converter

3. Shopping Discount Calculator

4. Student Grade Calculator

5. Library Late-Fee Calculator

6. Bus Fare Calculator

7. Daily Expense Tracker

8. Number Property Checker

9. Water Usage Bill

10. Multiplication Table Generator

How to use this report

Each case study includes a problem statement, objectives, assumptions, algorithm, text flowchart, pseudocode, Python

source code, test cases, sample output, and conclusion. The programs use introductory Python features such as

input/output, variables, arithmetic, conditions, lists, and loops.

Run each program and add screenshots of your own runs if required by your faculty. Review the code so you can explain

it confidently.

CASE STUDY 1: Simple Interest Calculator

1. Problem Statement

Write a Python program to calculate simple interest and the total amount payable from principal, annual rate, and time.

2. Objectives

Read principal, annual interest rate, and time from the user

Calculate simple interest using the standard formula.

Display the interest and total amount.

3. Assumptions

Principal, rate, and time must not be negative

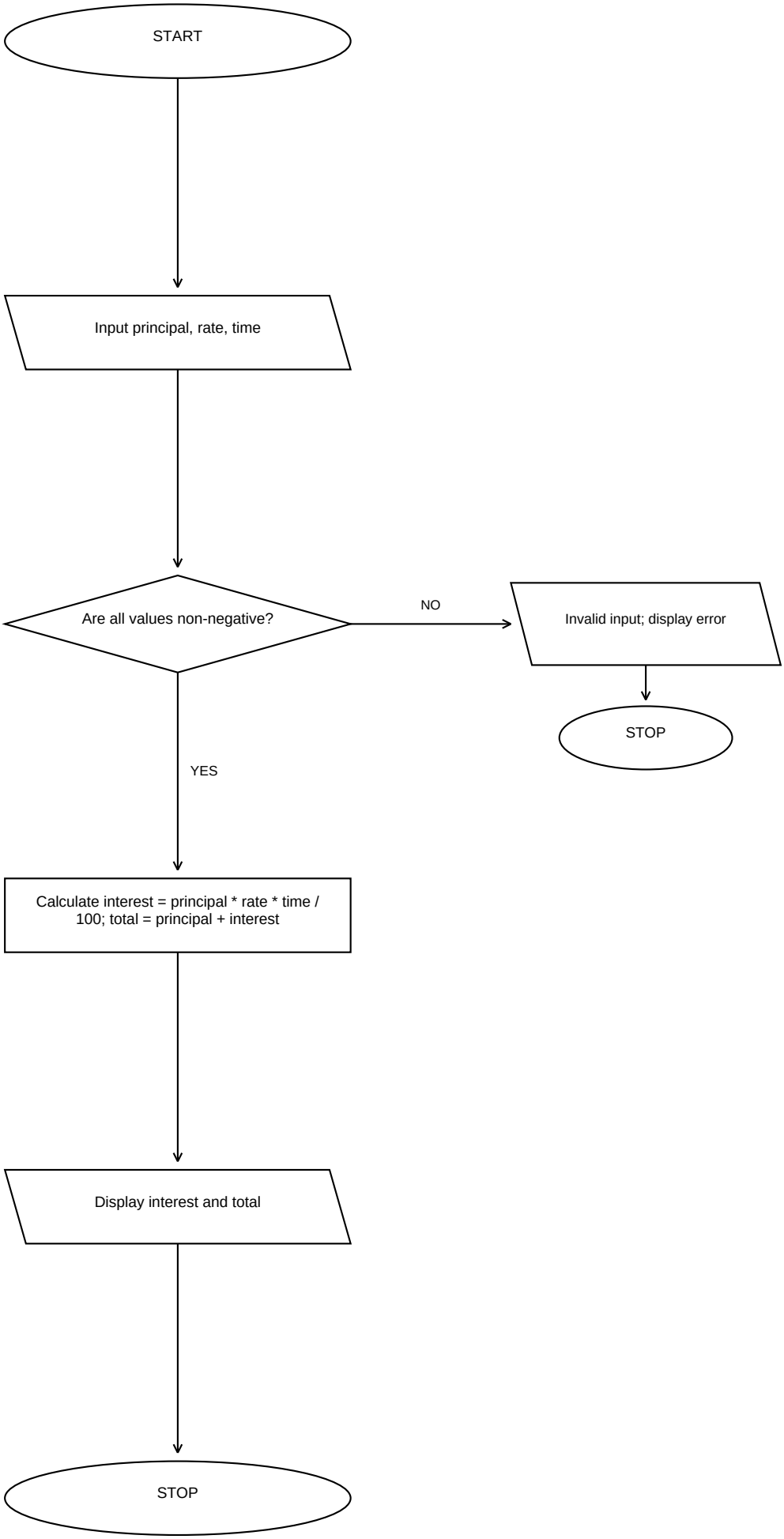
Simple Interest = (Principal × Rate × Time) / 100.

4. Algorithm

5. 1. Start.
6. 2. Read principal, rate, and time.
7. 3. Check that all three values are non-negative.
8. 4. Calculate simple interest and total amount.
9. 5. Display the results and stop.

CASE STUDY 1: Simple Interest Calculator

5. Flowchart



6. Pseudocode

START

INPUT principal, rate, time

IF any value is negative:

    PRINT error

ELSE:

    interest = principal * rate * time / 100

    total = principal + interest

    PRINT interest, total

STOP

7. Python Source Code

```
principal = float(input("Enter principal amount (Rs.): "))
rate = float(input("Enter annual rate (%): "))
time = float(input("Enter time (years): "))
if principal < 0 or rate < 0 or time < 0:
   print("Invalid input: values cannot be negative.")
else:
interest = principal * rate * time / 100
total = principal + interest
print("Simple interest: Rs.", round(interest, 2))
print("Total amount: Rs.", round(total, 2))
```

8. Test Cases

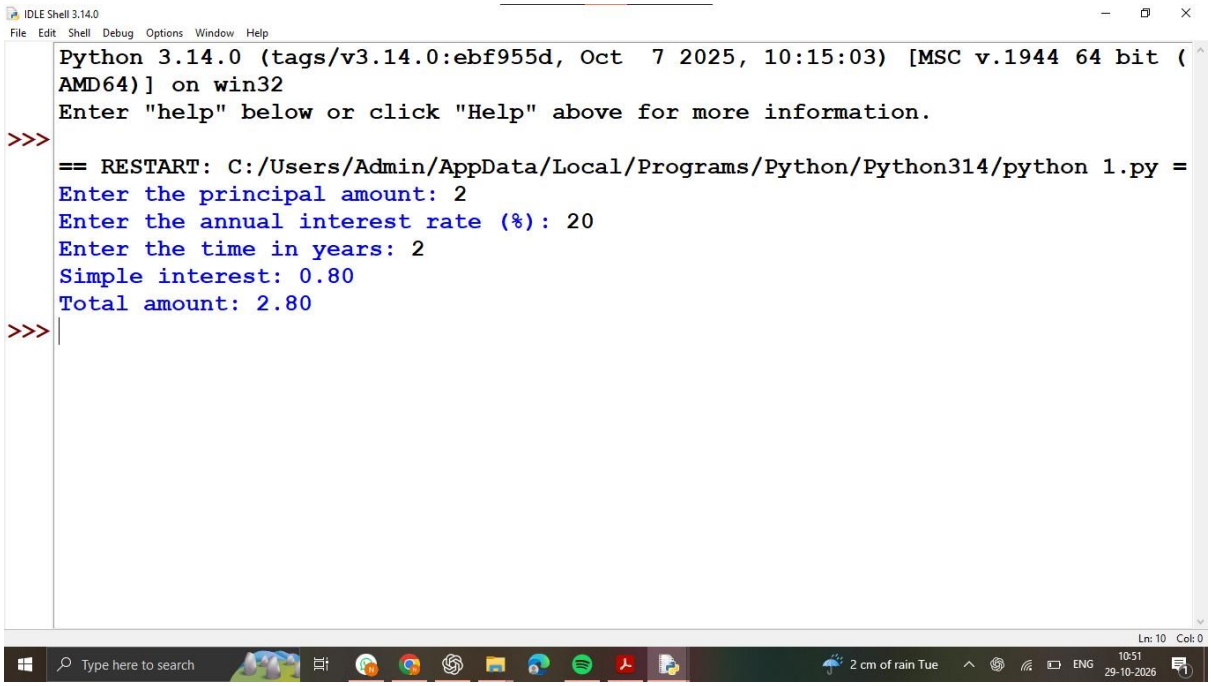
Test input / situation Expected result Purpose

10000, 5, 2 Interest n1000; total n11000 Normal

5000, 4, 1 Interest n200; total n5200 Alternate

-100, 5, 2 Invalid input message Invalid

9. Sample Output



10. Conclusion

This program practises input, arithmetic operators, floating-point values, and if-else validation.

CASE STUDY 2: Temperature Converter

1. Problem Statement

Create a program that converts a temperature from Celsius to Fahrenheit or Fahrenheit to Celsius based on the user's

choice.

2. Objectives

• Accept a conversion choice and temperature.

• Use the correct conversion formula.

• Display the converted temperature.

3. Assumptions

• Choice 1 means Celsius to Fahrenheit; choice 2 means Fahrenheit to Celsius.

• The temperature may be positive, zero, or negative.

4. Algorithm

1. Start.

2. Display the two conversion choices.

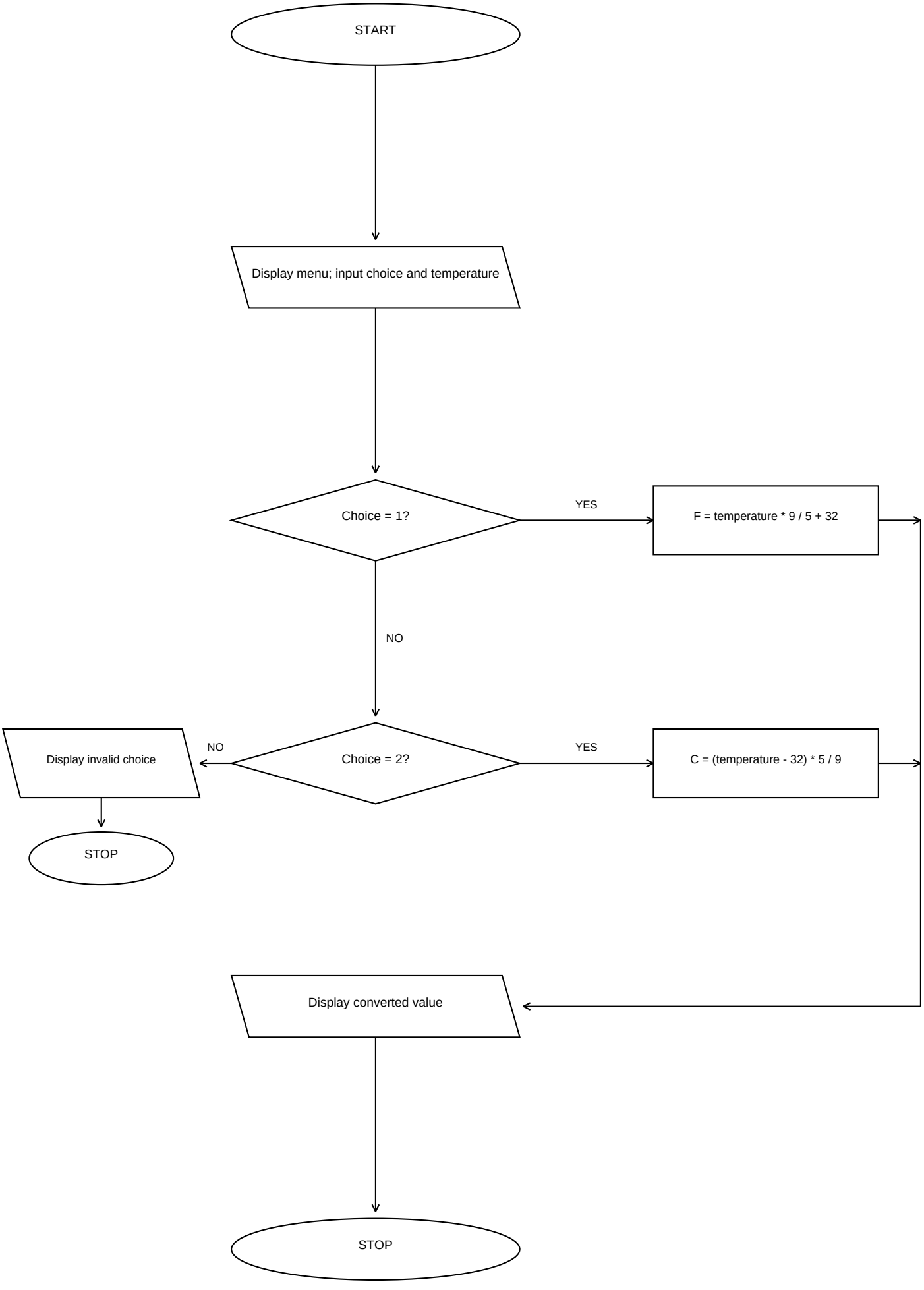
3. Read the choice and temperature.

4. Apply the matching formula or display an invalid-choice message.

5. Display the converted value and stop.

CASE STUDY 2: Temperature Converter

5. Flowchart



6. Pseudocode

START

PRINT conversion menu

INPUT choice, temperature

IF choice = 1:

    result = temperature * 9 / 5 + 32

ELSE IF choice = 2:

    result = (temperature - 32) * 5 / 9

ELSE:

    PRINT invalid choice

PRINT result

STOP

7. Python Source Code

```
print("1. Celsius to Fahrenheit")
print("2. Fahrenheit to Celsius")
choice = input("Choose 1 or 2: ")
temperature = float(input("Enter temperature: "))
if choice == "1":
   result = temperature * 9 / 5 + 32
   print("Temperature in Fahrenheit:", round(result, 2))
elif choice == "2":
result = (temperature - 32) * 5 / 9
print("Temperature in Celsius:", round(result, 2))
else:
print("Invalid choice.")
```

8. Test Cases

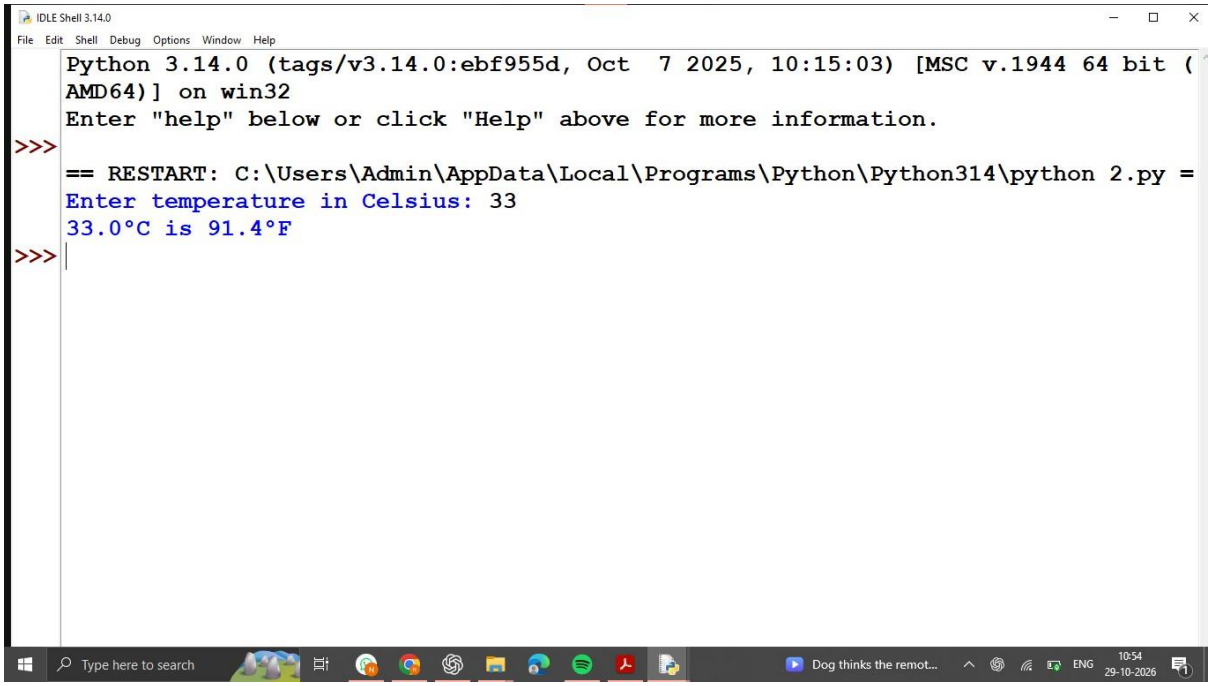
Test input / situation Expected result Purpose

1, 0 32°F Normal

2, 98.6 37°C approximately Alternate

choice 3 Invalid choice message Invalid

9.Sample Output



10. Conclusion

This program demonstrates menu-driven programming, arithmetic formulas, and conditional statements.

CASE STUDY 3: Shopping Discount Calculator

1. Problem Statement

Calculate a customer's discount and final shopping bill according to the total purchase amount.

2. Objectives

• Read the shopping amount.

• Select a discount rate using conditions.

• Display discount and final payable amount.

3. Assumptions

• Amount below n1000 gets no discount.

• Amount from n1000 to below n3000 gets 5% discount.

• Amount of n3000 or more gets 10% discount.

• Purchase amount cannot be negative.

4. Algorithm

1. Start and read purchase amount.

2. Reject a negative amount.

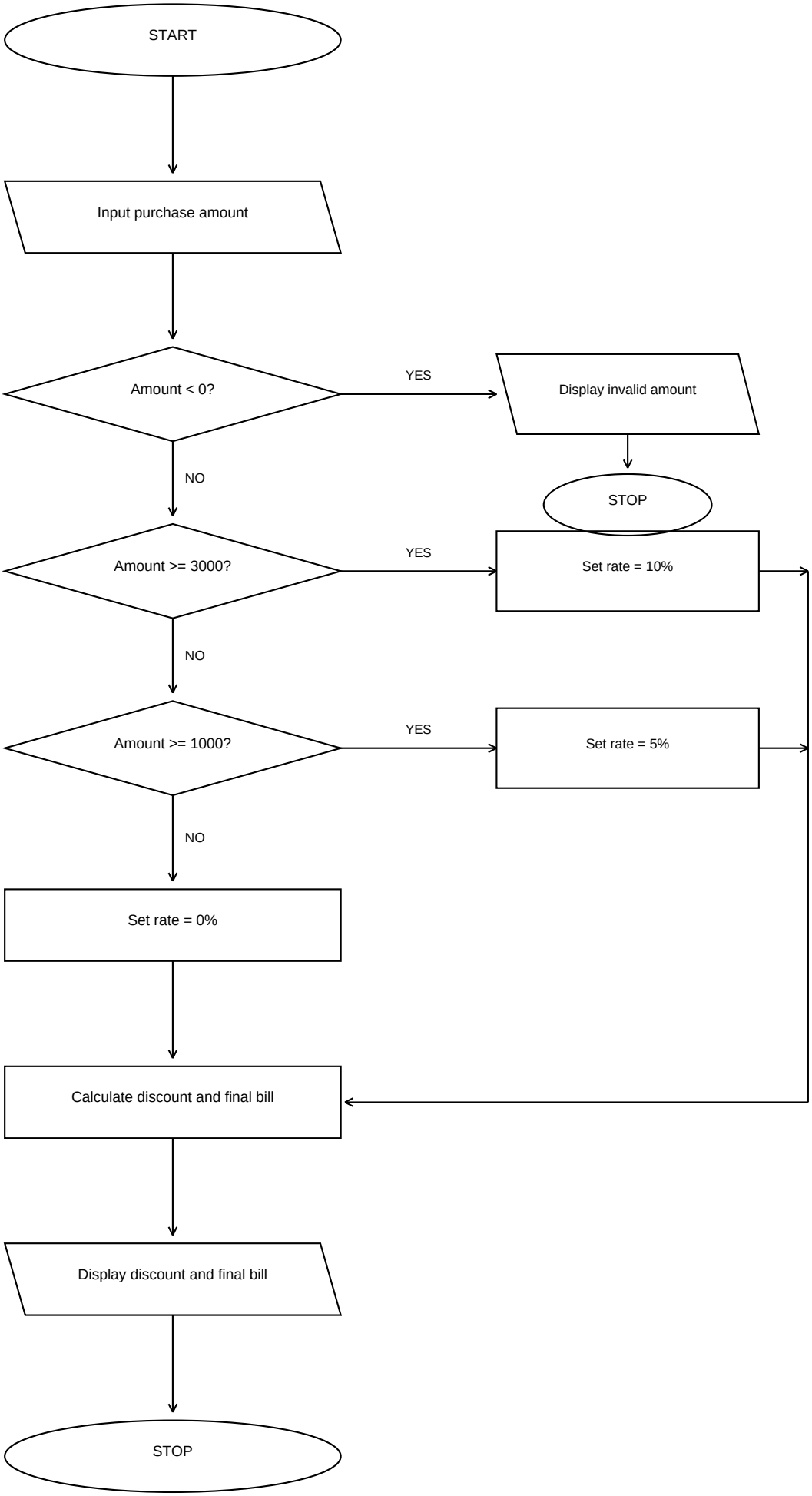
3. Choose the discount rate based on the amount.

4. Calculate discount and final bill.

5. Display both values and stop.

CASE STUDY 3: Shopping Discount Calculator

5. Flowchart



6. Pseudocode

```
START
INPUT amount
IF amount < 0:
    PRINT error
ELSE:
    IF amount >= 3000:
        rate = 10%
    ELSE IF amount >= 1000:
        rate = 5%
    ELSE:
        rate = 0%
    discount = amount * rate / 100
    final_bill = amount - discount
    PRINT discount, final_bill
STOP
```

7. Python Source Code

```
amount = float(input("Enter shopping amount (Rs.): "))
if amount < 0:
   print("Invalid amount.")
else:
if amount >= 3000:
   rate = 10
elif amount >= 1000:
rate = 5
else:
rate = 0
discount = amount * rate / 100
final_bill = amount - discount
print("Discount rate:", rate, "%")
print("Discount: Rs.", round(discount, 2))
print("Final bill: Rs.", round(final_bill, 2))
```

8. Test Cases

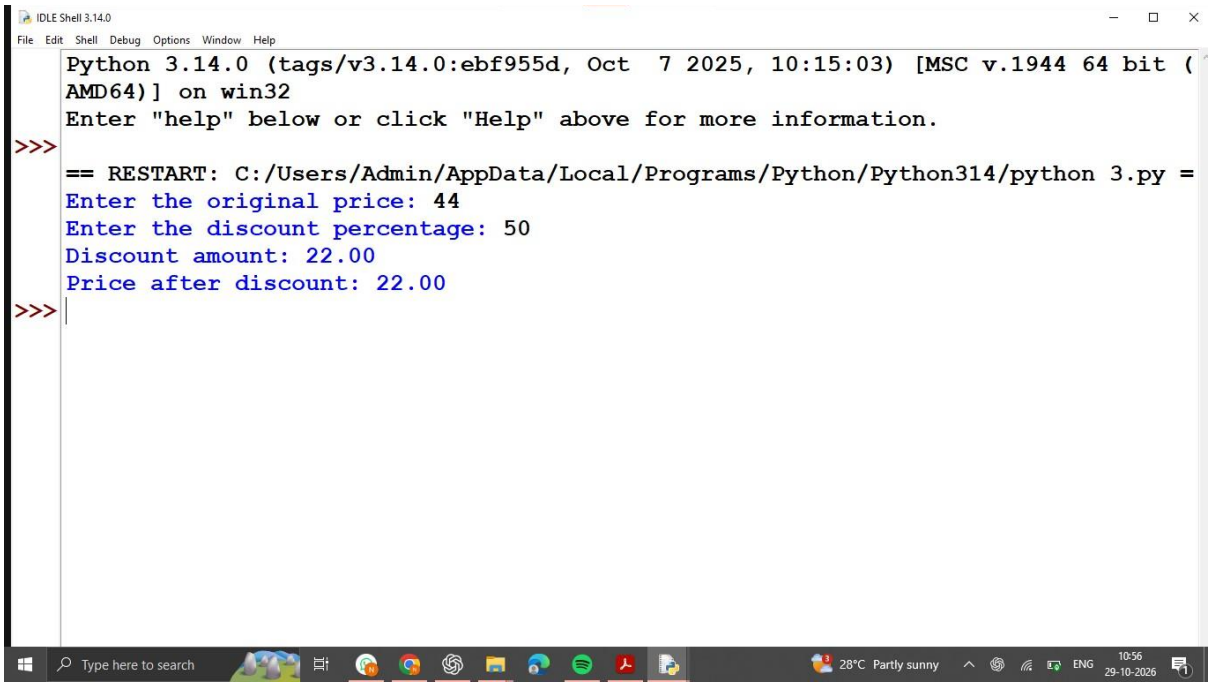
Test input / situation Expected result Purpose

800 No discount; final n800 Normal

2000 5% discount n100; final n1900 Alternate

4000 discount n400; final n3600 High purchase

9.Sample Output



10. Conclusion

This case study uses nested conditions and arithmetic to model a simple retail discount policy.

CASE STUDY 4: Student Grade Calculator

1. Problem Statement

Read marks in five subjects, calculate the average, and assign a grade based on the average.

2. Objectives

• Accept marks for five subjects.

• Validate that each mark is between 0 and 100.

• Calculate average and grade.

3. Assumptions

• Each subject is out of 100.

• Average 90 or above = A; 75–89.99 = B; 60–74.99 = C; 40–59.99 = D; below 40 = F.

4. Algorithm

1. Start and read five marks.

2. Check that all marks are within 0–100.

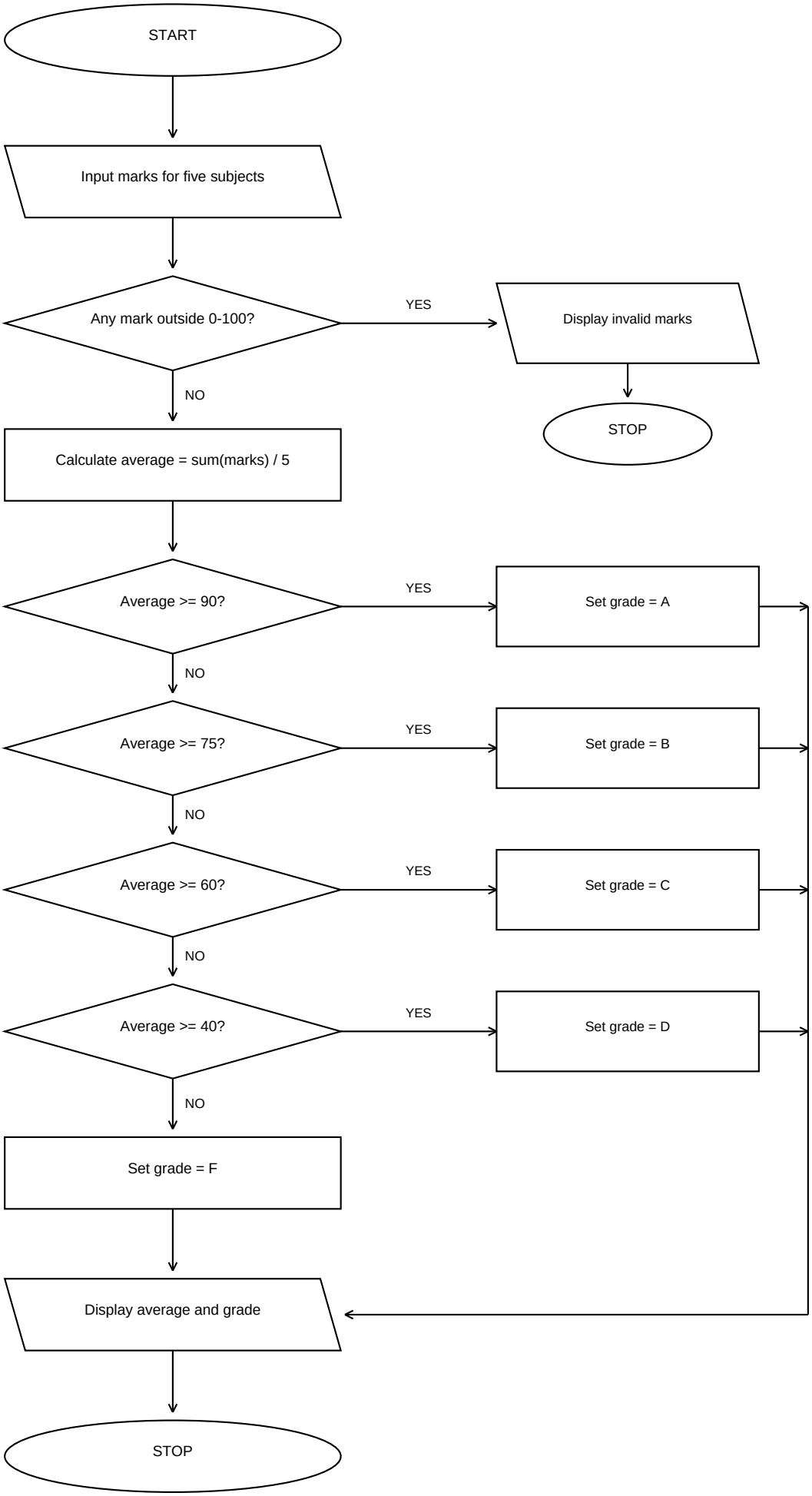
3. Calculate the average.

4. Use if-elif-else to assign a grade.

5. Display average and grade.

CASE STUDY 4: Student Grade Calculator

5. Flowchart



6. Pseudocode

```
START
INPUT five marks
IF any mark < 0 OR mark > 100:
    PRINT error
ELSE:
    average = sum(marks) / 5
    IF average >= 90:
        grade = A
    ELSE IF average >= 75:
        grade = B
    ELSE IF average >= 60:
        grade = C
    ELSE IF average >= 40:
        grade = D
    ELSE:
        grade = F
    PRINT average, grade
STOP
```

7. Python Source Code

```
marks = []
for i in range(1, 6):
   mark = float(input("Enter mark for subject " + str(i) + ": "))
   marks.append(mark)
   if any(mark < 0 or mark > 100 for mark in marks):
      print("Invalid marks: enter values from 0 to 100.")
   else:
   average = sum(marks) / len(marks)
   if average >= 90:
      grade = "A"
   elif average >= 75:
   grade = "B"
elif average >= 60:
grade = "C"
elif average >= 40:
grade = "D"
else:
grade = "F"
print("Average:", round(average, 2))
```

print("Grade:", grade)

8. Test Cases

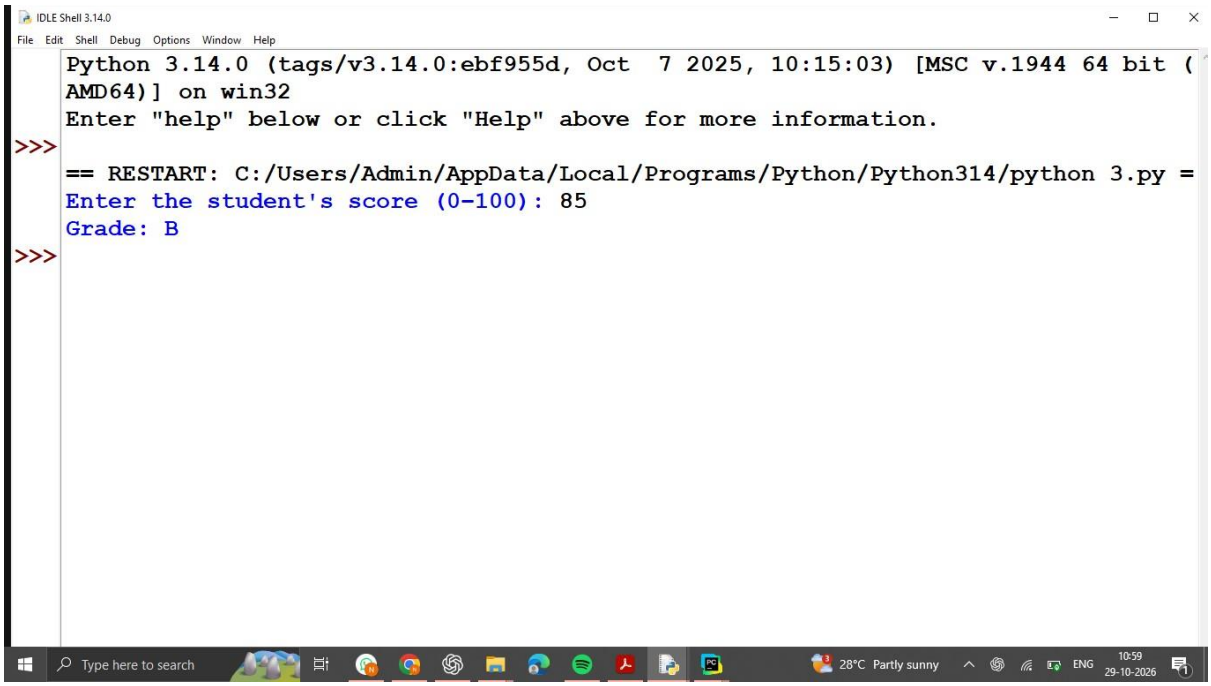
Test input / situation Expected result Purpose

95, 90, 92, 88, 95 Average 92; grade A Normal

70, 65, 75, 60, 80 Average 70; grade C Alternate

105 any subject Invalid marks message Invalid

9.Sample Output



10. Conclusion

This program introduces lists, loops, sum(), average calculation, and grade classification

CASE STUDY 5: Library Late-Fee Calculator

1. Problem Statement

Calculate a library late fee based on the number of days a book is returned after its due date.

2. Objectives

• Read the number of late days.

• Calculate a fee using simple daily slabs.

• Handle books returned on time and invalid input.

3. Assumptions

• No fee for 0 late days.

• n2 per day for days 1–5.

• n5 per day for each day after day 5, in addition to the first five days' fee.

• Late days cannot be negative.

4. Algorithm

1. Start and read late days.

2. If late days are negative, show an error.

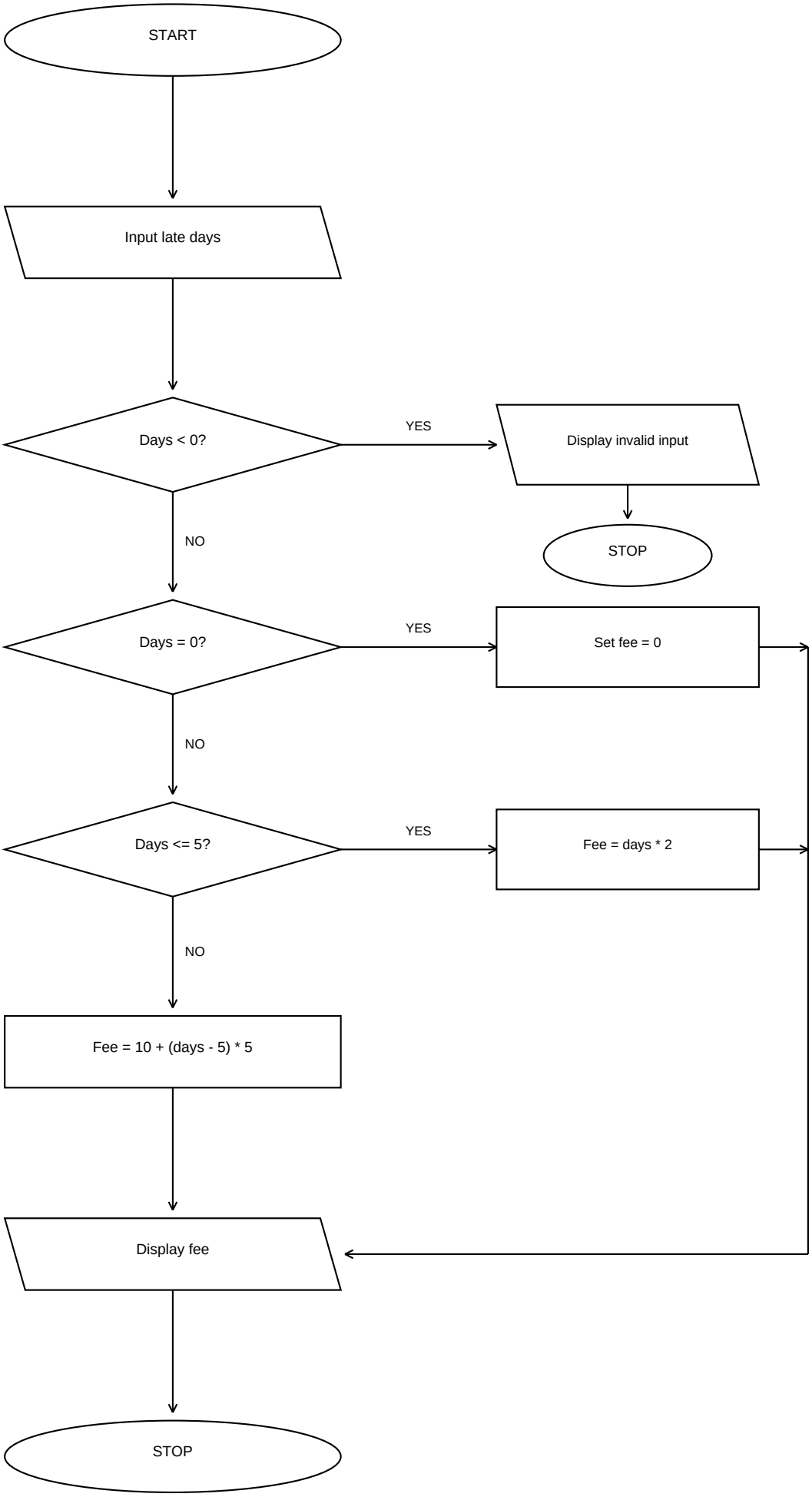
3. If 0, fee is zero; if 1–5, fee is days × n2.

4. If above 5, fee is n10 plus n5 for each extra day.

5. Display the fee and stop.

CASE STUDY 5: Library Late-Fee Calculator

5. Flowchart



6. Pseudocode

```
START
INPUT days
IF days < 0:
    PRINT error
ELSE IF days = 0:
    fee = 0
ELSE IF days <= 5:
    fee = days * 2
ELSE:
    fee = 5 * 2 + (days - 5) * 5
PRINT fee
STOP
```

7. Python Source Code

```
days = int(input("Enter number of late days: "))
if days < 0:
   print("Invalid input: days cannot be negative.")
elif days == 0:
fee = 0
elif days <= 5:
fee = days * 2
else:
fee = 5 * 2 + (days - 5) * 5
if days >= 0:
   print("Library late fee: Rs.", fee)
```

8. Test Cases

Test input / situation Expected result Purpose

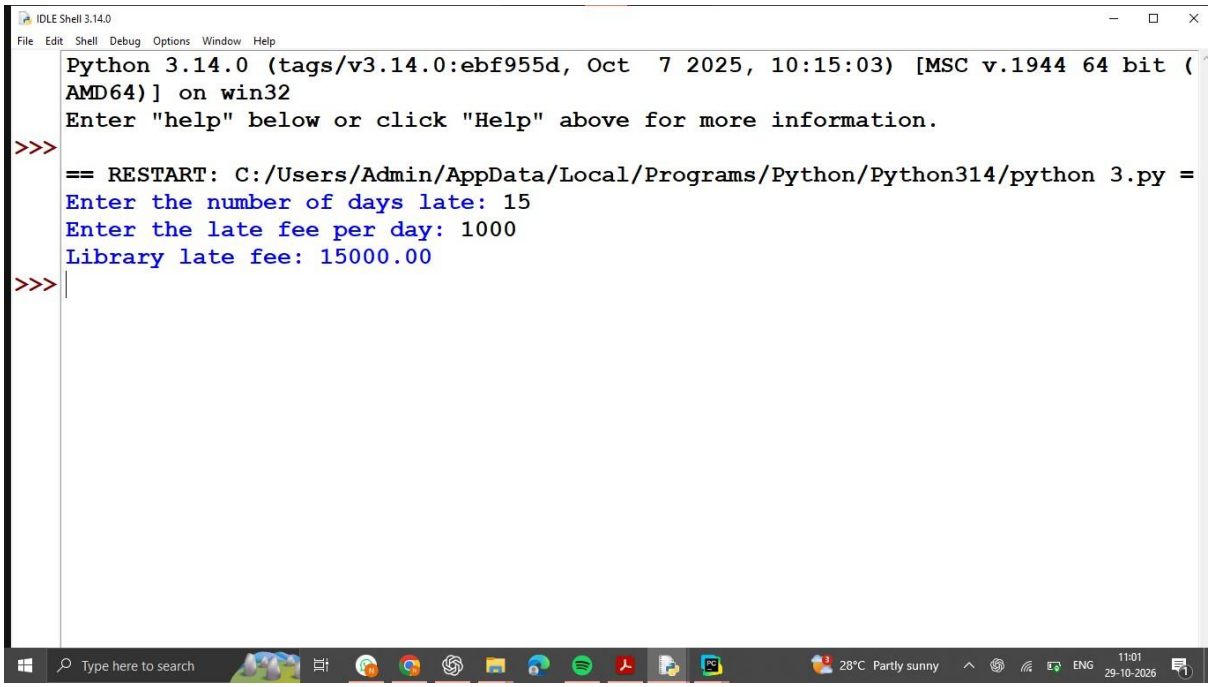
0 Fee n0 On time

4 Fee n8 Normal

8 Fee n25 Late return

-2 Invalid input message Invalid

9. Sample Output



10. Conclusion

The program practises integer input, decision-making, and calculations using fee slabs.

CASE STUDY 6: Bus Fare Calculator

1. Problem Statement

Calculate a passenger's bus fare based on travel distance, with a minimum fare and a per-kilometre rate.

2. Objectives

- **Accept the distance travelled.**
- **Calculate fare from the given rate.**
- **Display the total fare.**

3. Assumptions

- **The fare is n10 for the first kilometre or any shorter trip.**
- **For distance above 1 km, add n5 for each additional kilometre.**
- **Distance cannot be negative.**

4. Algorithm

1. Start and read distance.

2. Reject negative distance.

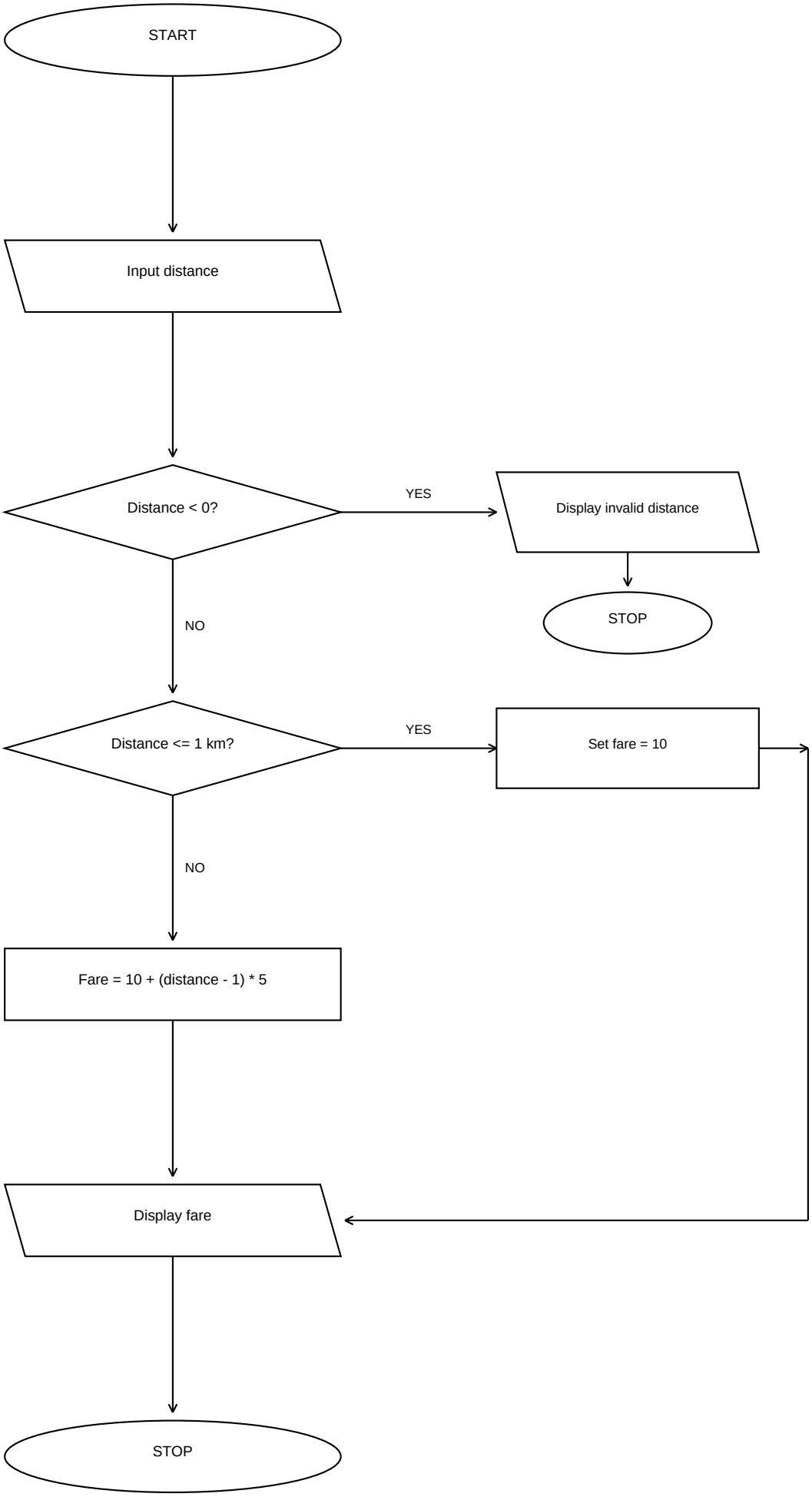
3. If distance is at most 1 km, fare is n10.

4. Otherwise calculate n10 plus n5 multiplied by the distance beyond 1 km.

5. Display fare and stop.

CASE STUDY 6: Bus Fare Calculator

5. Flowchart



6. Pseudocode

START

INPUT distance

IF distance < 0:

    PRINT error

ELSE IF distance <= 1:

    fare = 10

ELSE:

    fare = 10 + (distance - 1) * 5

PRINT fare

STOP

7. Python Source Code

```
distance = float(input("Enter distance in kilometres: "))
if distance < 0:
   print("Invalid distance.")
elif distance <= 1:
fare = 10
print("Bus fare: Rs.", fare)
else:
fare = 10 + (distance - 1) * 5
print("Bus fare: Rs.", round(fare, 2))
```

8. Test Cases

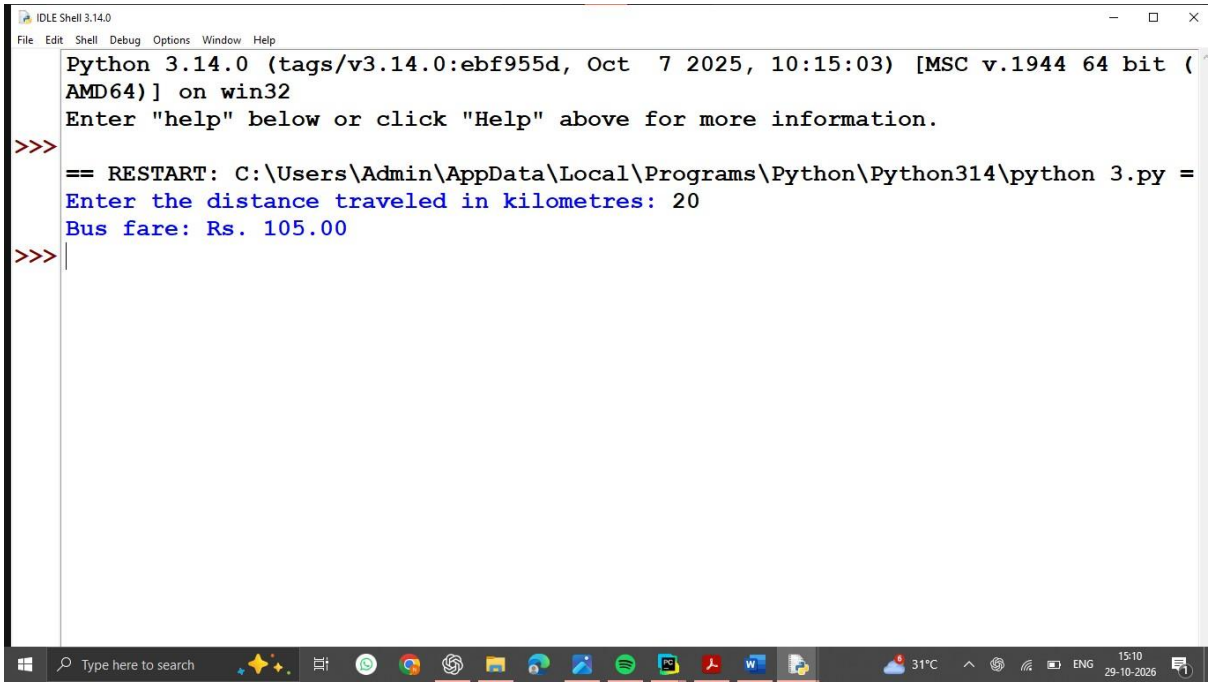
Test input / situation Expected result Purpose

0.5 km Fare n10 Short trip

1 km Fare n10 Boundary

5 km Fare n30 Longer trip

9.Sample Output



10. Conclusion

This program demonstrates conditional fare calculation using a simple transport pricing rule.

CASE STUDY 7: Daily Expense Tracker

1. Problem Statement

Record expenses for a day, calculate the total spent, and compare the total with a daily budget.

2. Objectives

- **Read a daily budget.**
- **Accept multiple expense amounts using a loop.**
- **Calculate the total and report whether the budget was exceeded.**

3. Assumptions

- **The user enters the number of expenses first.**
- **The number of expenses and all amounts must be non-negative.**
- **The budget is not exceeded when total spending equals the budget.**

4. Algorithm

1. Start and read the budget and number of expenses.

2. Validate both values.

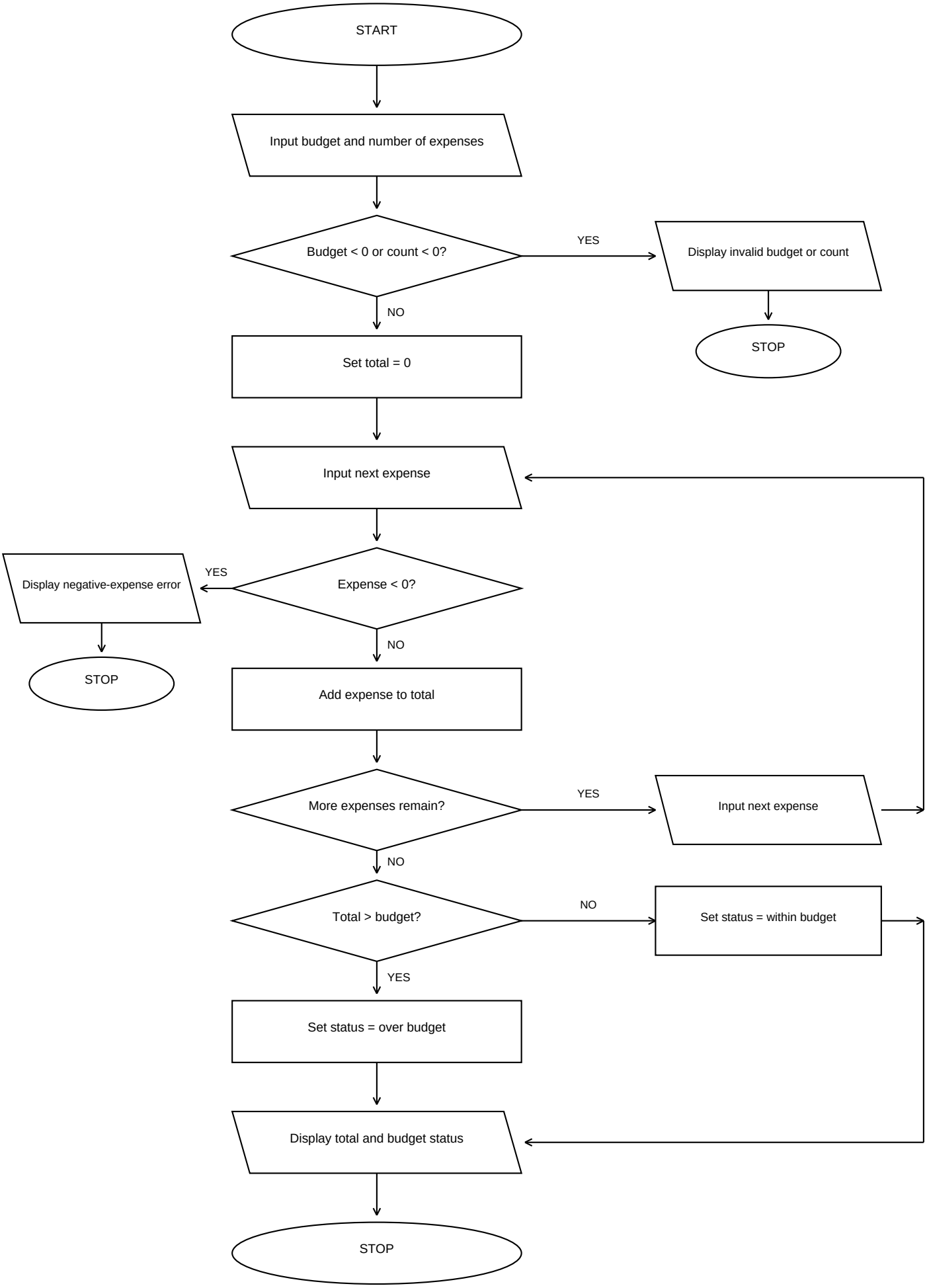
3. Use a loop to read each expense and add it to the total.

4. Compare total spending with the budget.

5. Display total and budget status.

CASE STUDY 7: Daily Expense Tracker

5. Flowchart



6. Pseudocode

```
START
INPUT budget, count
IF budget < 0 OR count < 0:
    PRINT error
ELSE:
    total = 0
    REPEAT count times:
        INPUT expense
        total = total + expense
    PRINT total
    IF total > budget:
        PRINT over budget
    ELSE:
        PRINT within budget
STOP
```

7. Python Source Code

```
budget = float(input("Enter daily budget (Rs.): "))
count = int(input("How many expenses? "))
if budget < 0 or count < 0:
   print("Invalid budget or number of expenses.")
else:
total = 0
for i in range(count):
   expense = float(input("Enter expense amount (Rs.): "))
   if expense < 0:
      print("Expense cannot be negative.")
      total = None
      break
      total += expense
      if total is not None:
         print("Total spending: Rs.", round(total, 2))
         if total > budget:
            print("Status: Budget exceeded.")
```

```
print("Status: Within budget.")
```

8. Test Cases

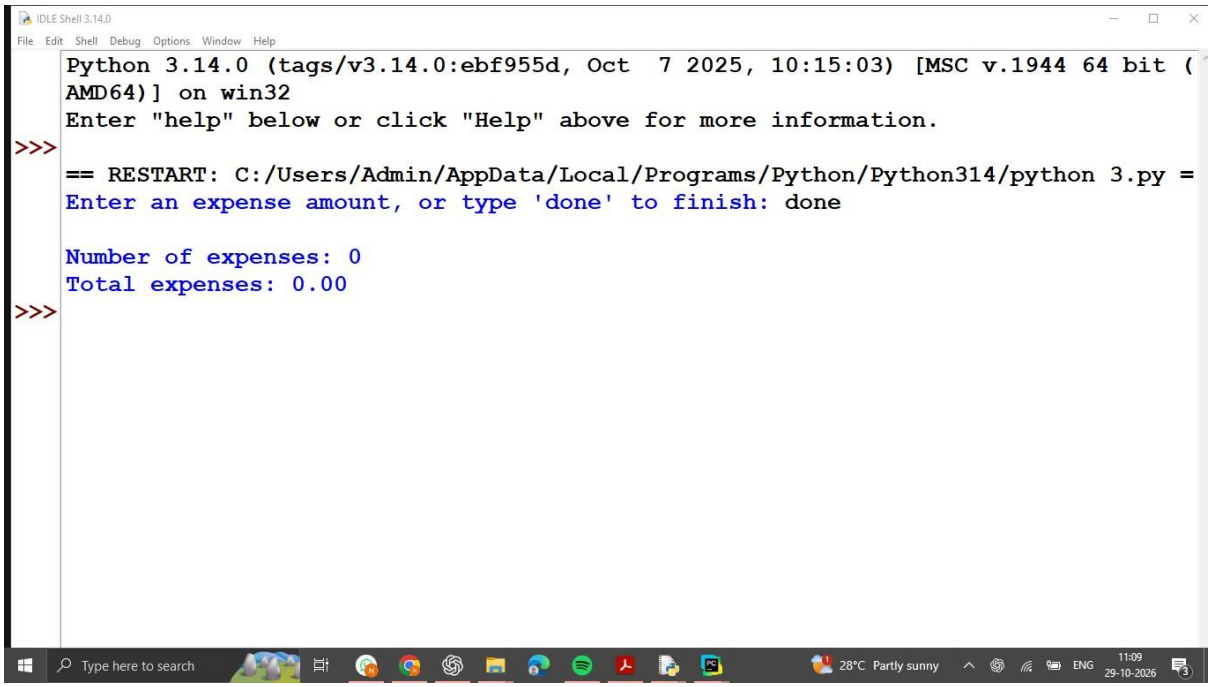
Test input / situation Expected result Purpose

Budget 500; expenses 100, 150 Total n250; within budget Normal

Budget 300; expenses 200, 150 Total n350; budget exceeded Over budget

Negative expense Error message Invalid

9.Sample Output



10. Conclusion

The case study uses a for loop, a running total, and conditional statements for basic budgeting

CASE STUDY 8: Number Property Checker

1. Problem Statement

Check whether an integer is positive, negative, or zero, and determine whether it is even or odd.

2. Objectives

- **Read an integer.**
- **Classify the number by sign.**
- **Use the modulus operator to check even or odd.**

3. Assumptions

- **The input must be an integer.**
- **Zero is neither positive nor negative, and zero is even.**

4. Algorithm

1. Start and read an integer.

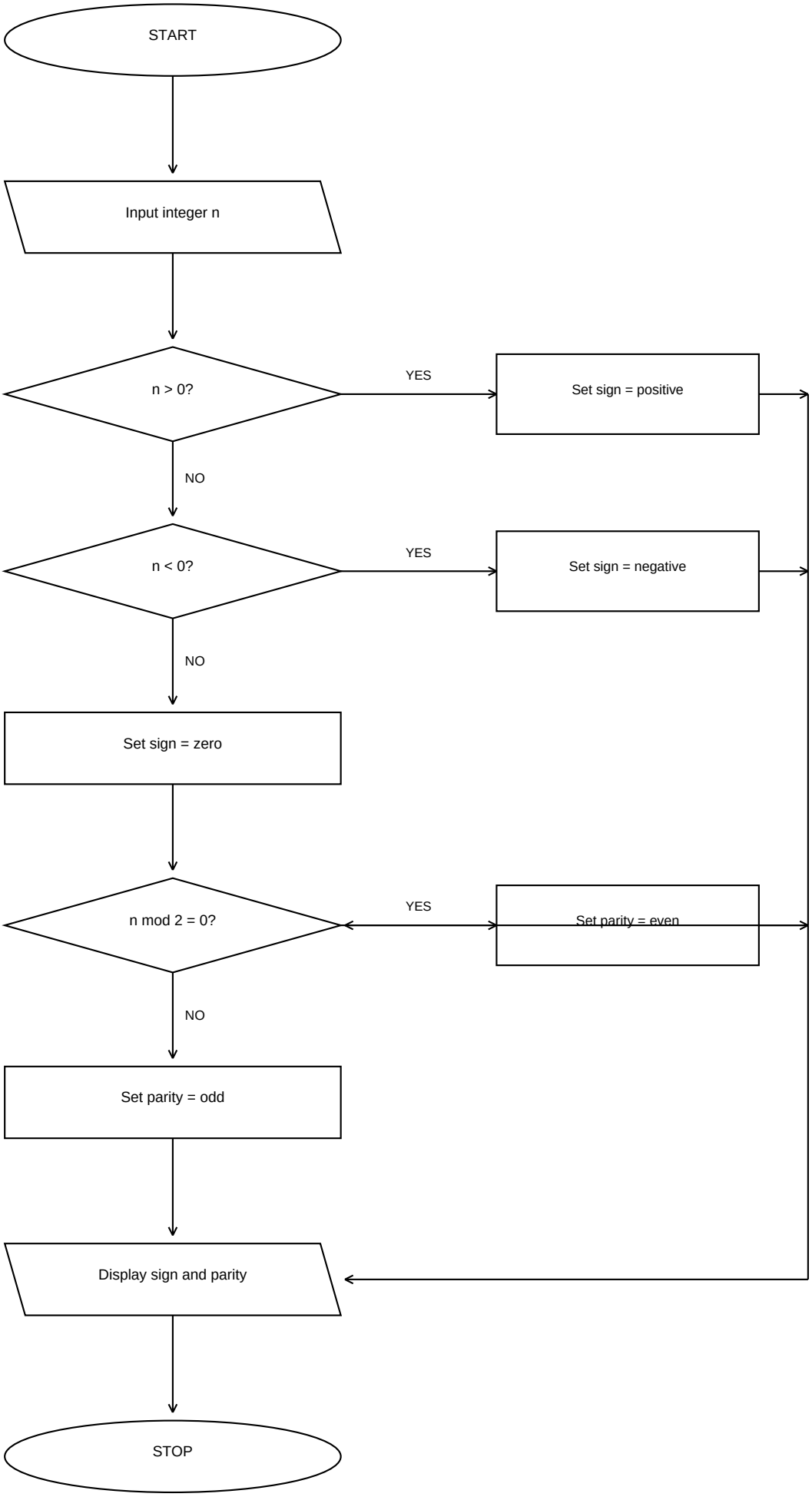
2. Check whether it is greater than, less than, or equal to zero.

3. Check remainder after division by 2.

4. Display sign and parity.

CASE STUDY 8: Number Property Checker

5. Flowchart



6. Pseudocode

START

INPUT n

IF n > 0:

    PRINT positive

ELSE IF n < 0:

    PRINT negative

ELSE:

    PRINT zero

IF n % 2 = 0:

    PRINT even

ELSE:

    PRINT odd

STOP

7. Python Source Code

```
number = int(input("Enter an integer: "))
if number > 0:
   print("The number is positive.")
elif number < 0:
print("The number is negative.")
else:
print("The number is zero.")
if number % 2 == 0:
   print("The number is even.")
else:
print("The number is odd.")
```

8. Test Cases

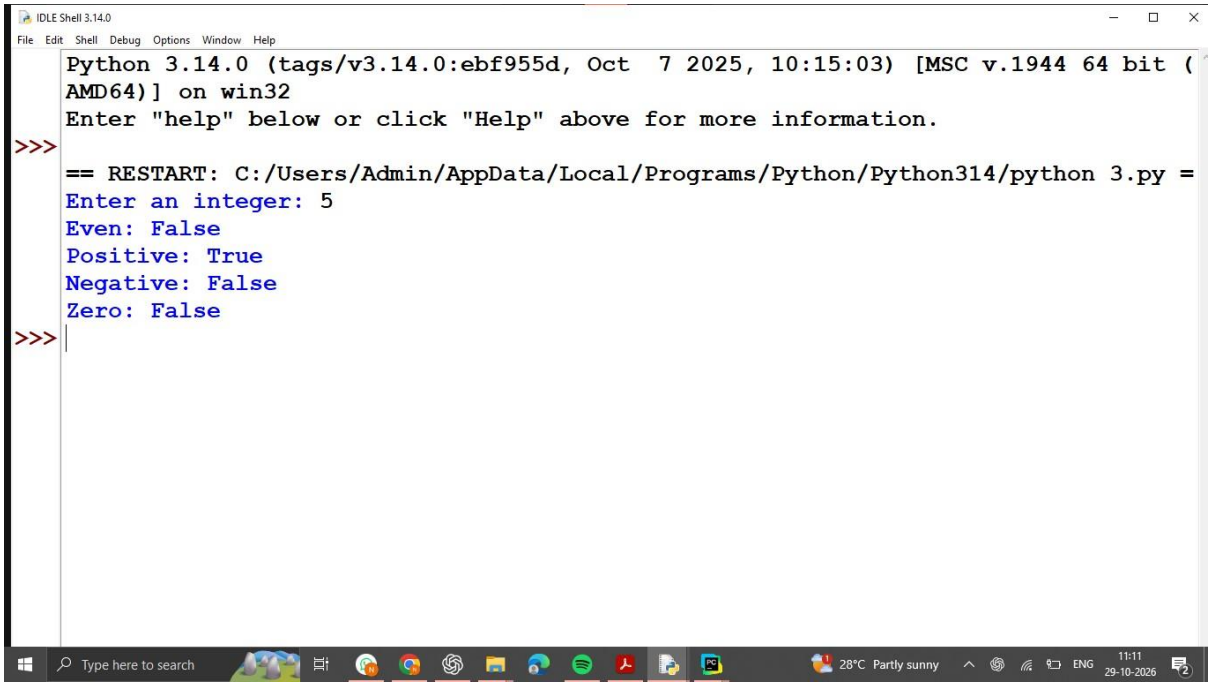
Test input / situation Expected result Purpose

7 Positive; odd Positive odd

-4 Negative; even Negative even

0 Zero; even Zero

9.Sample Output



10. Conclusion

This beginner program uses comparison operators, if-elif-else, and the modulus operator.

CASE STUDY 9: Water Usage Bill

1. Problem Statement

Calculate a sample household water bill using a basic per-unit charge and a fixed service fee.

2. Objectives

- **Read water units consumed.**
- **Calculate usage charge and fixed fee.**
- **Display a bill summary.**

3. Assumptions

- **The first 20 units cost n3 per unit.**
- **Units above 20 cost n5 per unit.**
- **A fixed service fee of n30 is added.**
- **Units cannot be negative.**

4. Algorithm

1. Start and read units consumed.

2. Reject negative units.

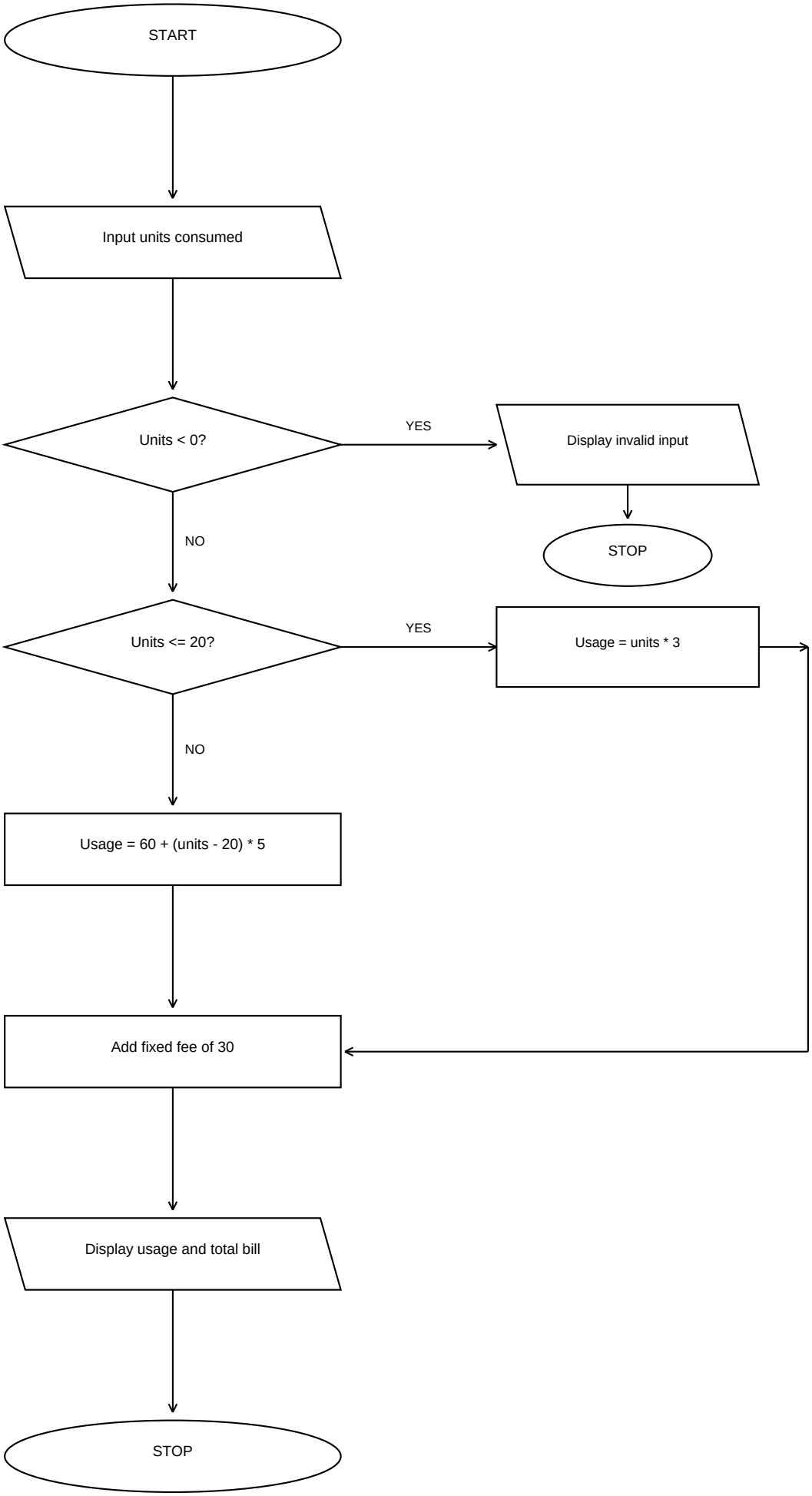
3. Calculate the charge using the two slabs.

4. Add the fixed fee.

5. Display usage charge and final bill.

CASE STUDY 9: Water Usage Bill

5. Flowchart



6. Pseudocode

```
START
INPUT units
IF units < 0:
    PRINT error
ELSE IF units <= 20:
    usage = units * 3
ELSE:
    usage = 20 * 3 + (units - 20) * 5
    bill = usage + 30
    PRINT usage, bill
STOP
```

7. Python Source Code

```
units = int(input("Enter water units consumed: "))
if units < 0:
   print("Invalid input: units cannot be negative.")
else:
if units <= 20:
   usage_charge = units * 3
else:
usage_charge = 20 * 3 + (units - 20) * 5
fixed_fee = 30
total_bill = usage_charge + fixed_fee
print("Usage charge: Rs.", usage_charge)
print("Fixed service fee: Rs.", fixed_fee)
print("Total water bill: Rs.", total_bill)
```

8. Test Cases

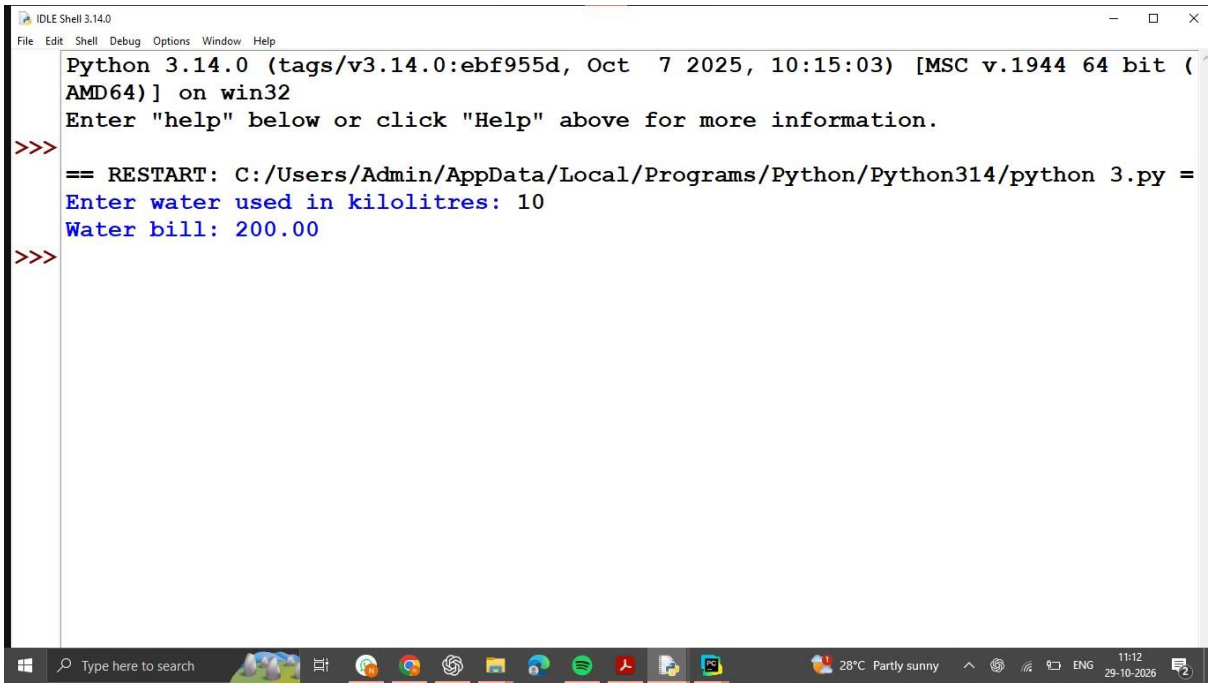
Test input / situation Expected result Purpose

10 units Usage n30; total n60 Normal

25 units Usage n85; total n115 Above first slab

-1 unit Invalid input message Invalid

9. Sample Output



10. Conclusion

This program applies arithmetic and conditional slabs to a household utility-billing example.

CASE STUDY 10: Multiplication Table Generator

1. Problem Statement

Generate the multiplication table of a number up to a user-selected limit.

2. Objectives

- **Read a number and a limit.**
- **Use a loop to generate multiplication results.**
- **Validate the limit.**

3. Assumptions

- **The table limit must be between 1 and 20.**
- **The number may be positive, negative, or zero.**

4. Algorithm

1. Start and read number and limit.

2. Check that limit is from 1 to 20.

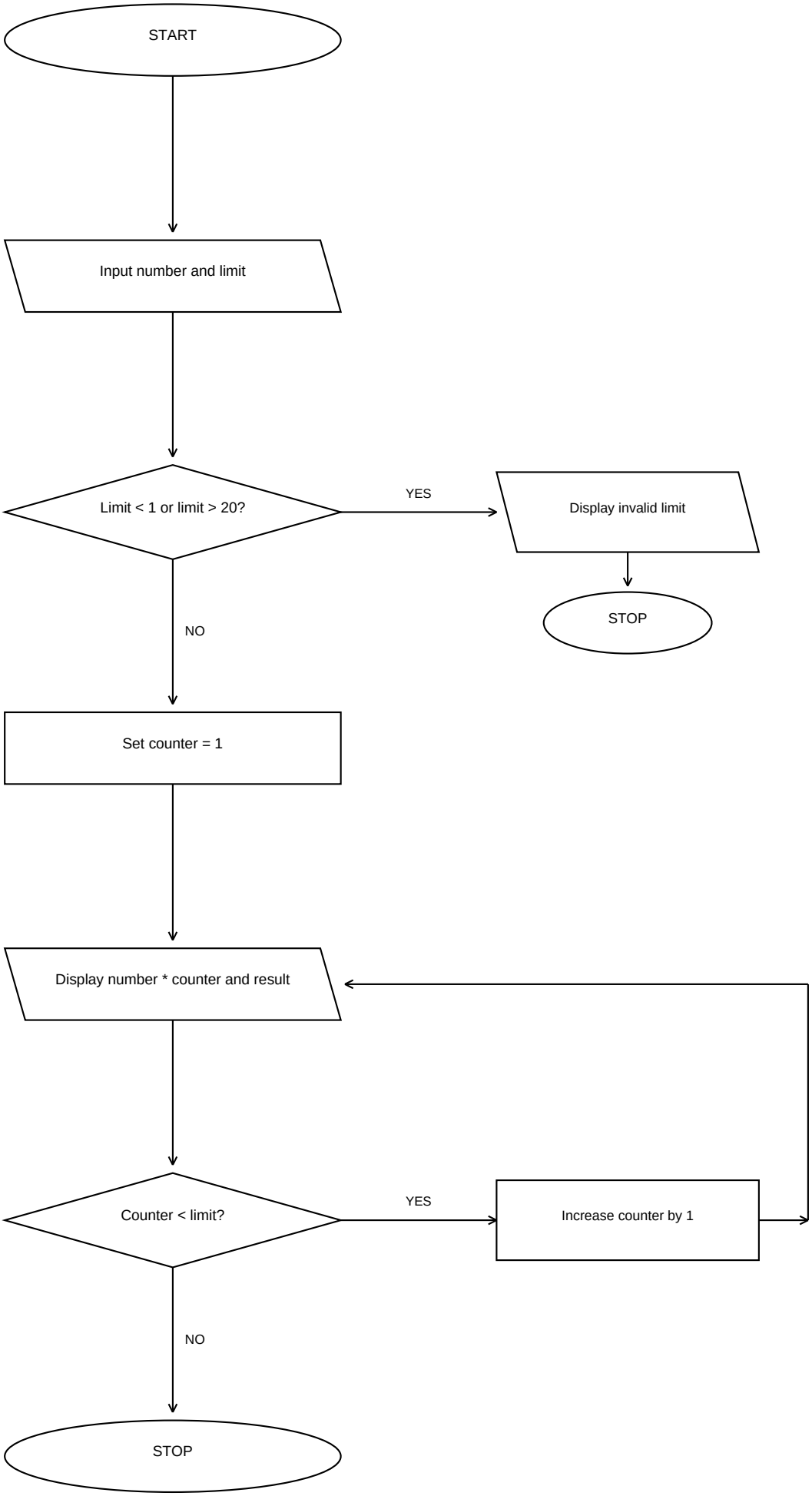
3. Use a for loop from 1 to the limit.

4. Print number × counter and its result.

5. Stop.

CASE STUDY 10: Multiplication Table Generator

5. Flowchart



6. Pseudocode

```
START
INPUT number, limit
IF limit < 1 OR limit > 20:
    PRINT error
ELSE:
    FOR i from 1 to limit:
        PRINT number, '*', i, '=', number * i
STOP
```

7. Python Source Code

```
number = int(input("Enter a number: "))
limit = int(input("Enter table limit (1 to 20): "))
if limit < 1 or limit > 20:
   print("Invalid limit. Enter a value from 1 to 20.")
else:
for i in range(1, limit + 1):
   print(number, "x", i, "=", number * i)
```

8. Test Cases

Test input / situation Expected result Purpose

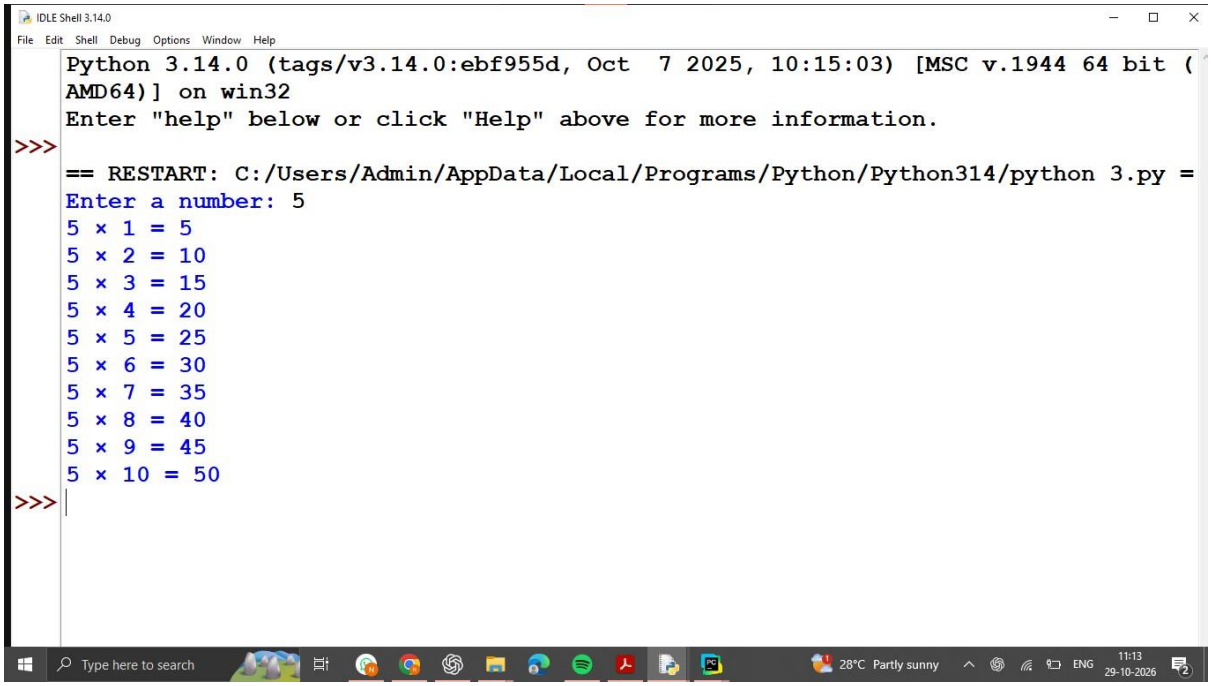
Number 3, limit 5 3 x 1 = 3 through 3 x 5 = 15 Normal

Number 7, limit 1 7 x 1 = 7 Boundary

Test input / situation Expected result Purpose

Limit 0 Invalid limit message Invalid

9.Sample Output



10. Conclusion

The program demonstrates for loops, range(), multiplication, and simple input validation.